

```
!wget http://www.cs.cornell.edu/~cristian/data/cornell_movie_dialogs_corpus.zip
!unzip cornell_movie_dialogs_corpus.zip
```

```
--2025-11-23 14:48:01-- http://www.cs.cornell.edu/~cristian/data/cornell_movie_dialogs_corpus.zip
Resolving www.cs.cornell.edu (www.cs.cornell.edu)... 132.236.207.53
Connecting to www.cs.cornell.edu (www.cs.cornell.edu)|132.236.207.53|:80... connected.
HTTP request sent, awaiting response... 301 Moved Permanently
Location: https://www.cs.cornell.edu/~cristian/data/cornell_movie_dialogs_corpus.zip [following]
--2025-11-23 14:48:01-- https://www.cs.cornell.edu/~cristian/data/cornell_movie_dialogs_corpus.zip
Connecting to www.cs.cornell.edu (www.cs.cornell.edu)|132.236.207.53|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 9916637 (9.5M) [application/zip]
Saving to: 'cornell_movie_dialogs_corpus.zip'
```

```
cornell_movie_dialo 100%[=====>] 9.46M 36.7MB/s in 0.3s
```

```
2025-11-23 14:48:01 (36.7 MB/s) - 'cornell_movie_dialogs_corpus.zip' saved [9916637/9916637]
```

```
Archive: cornell_movie_dialogs_corpus.zip
  creating: cornell movie-dialogs corpus/
  inflating: cornell movie-dialogs corpus/.DS_Store
   creating: __MACOSX/
   creating: __MACOSX/cornell movie-dialogs corpus/
  inflating: __MACOSX/cornell movie-dialogs corpus/.DS_Store
  inflating: cornell movie-dialogs corpus/chameleons.pdf
  inflating: __MACOSX/cornell movie-dialogs corpus/.chameleons.pdf
  inflating: cornell movie-dialogs corpus/movie_characters_metadata.txt
  inflating: cornell movie-dialogs corpus/movie_conversations.txt
  inflating: cornell movie-dialogs corpus/movie_lines.txt
  inflating: cornell movie-dialogs corpus/movie_titles_metadata.txt
  inflating: cornell movie-dialogs corpus/raw_script_urls.txt
  inflating: cornell movie-dialogs corpus/README.txt
  inflating: __MACOSX/cornell movie-dialogs corpus/.README.txt
```

```
# 0.1 Install required libraries (run once)
!pip install -q transformers datasets accelerate sentencepiece

# check GPU
!nvidia-smi
```

```
/bin/bash: line 1: nvidia-smi: command not found
```

```
# list folder
!ls -la "cornell movie-dialogs corpus"
!sed -n '1,5p' "cornell movie-dialogs corpus/movie_lines.txt"
!sed -n '1,5p' "cornell movie-dialogs corpus/movie_conversations.txt"
```

```
total 41560
drwxr-xr-x 2 root root 4096 May 10 2011 .
drwxr-xr-x 1 root root 4096 Nov 23 14:48 ..
-rw-r--r-- 1 root root 290691 May 9 2011 chameleons.pdf
-rw-r--r-- 1 root root 6148 May 10 2011 .DS_Store
-rw-r--r-- 1 root root 705695 Dec 18 2010 movie_characters_metadata.txt
-rw-r--r-- 1 root root 6760930 Dec 17 2010 movie_conversations.txt
-rw-r--r-- 1 root root 34641919 Dec 17 2010 movie_lines.txt
-rw-r--r-- 1 root root 67289 Dec 17 2010 movie_titles_metadata.txt
-rw-r--r-- 1 root root 56177 Dec 18 2010 raw_script_urls.txt
-rw-r--r-- 1 root root 4182 May 10 2011 README.txt
```

```

L1045 +++$+++ u0 +++$+++ m0 +++$+++ BIANCA +++$+++ They do not!
L1044 +++$+++ u2 +++$+++ m0 +++$+++ CAMERON +++$+++ They do to!
L985 +++$+++ u0 +++$+++ m0 +++$+++ BIANCA +++$+++ I hope so.
L984 +++$+++ u2 +++$+++ m0 +++$+++ CAMERON +++$+++ She okay?
L925 +++$+++ u0 +++$+++ m0 +++$+++ BIANCA +++$+++ Let's go.
u0 +++$+++ u2 +++$+++ m0 +++$+++ ['L194', 'L195', 'L196', 'L197']
u0 +++$+++ u2 +++$+++ m0 +++$+++ ['L198', 'L199']
u0 +++$+++ u2 +++$+++ m0 +++$+++ ['L200', 'L201', 'L202', 'L203']
u0 +++$+++ u2 +++$+++ m0 +++$+++ ['L204', 'L205', 'L206']
u0 +++$+++ u2 +++$+++ m0 +++$+++ ['L207', 'L208']

```

```

import re, csv
from pathlib import Path

def clean_text(s: str) -> str:
    s = s.strip().lower()
    s = re.sub(r"\s+", " ", s)
    # keep common punctuation
    s = re.sub(r"^[^a-z0-9?!.,\'\\"/:;()\- ]+", "", s)
    return s

lines_path = Path("cornell movie-dialogs corpus") / "movie_lines.txt"
convs_path = Path("cornell movie-dialogs corpus") / "movie_conversations.txt"

# load lines
id2line = {}
with open(lines_path, encoding="latin-1") as f:
    for line in f:
        parts = line.split(" +++$+++ ")
        if len(parts) >= 5:
            line_id = parts[0].strip()
            text = parts[-1].strip()
            text = clean_text(text)
            id2line[line_id] = text

pairs = []
with open(convs_path, encoding="latin-1") as f:
    for line in f:
        parts = line.split(" +++$+++ ")
        if len(parts) >= 4:
            ids = eval(parts[-1].strip())
            for i in range(len(ids)-1):
                a = id2line.get(ids[i], "")
                b = id2line.get(ids[i+1], "")
                if a and b:
                    # filter very short lines
                    if len(a.split()) >= 1 and len(b.split()) >= 1:
                        pairs.append((a, b))

len(pairs)
# write to CSV (limit if needed, e.g. 100k)
out_csv = "cornell_pairs.csv"
max_pairs = 100000
with open(out_csv, "w", encoding="utf-8", newline="") as csvf:
    writer = csv.writer(csvf)
    writer.writerow(["input", "response"])
    for i, (a, b) in enumerate(pairs):
        if i >= max_pairs:
            break
    writer.writerow([a, b])

```



```
print("Saved", out_csv)
```

Saved cornell_pairs.csv

```
import pandas as pd
df = pd.read_csv("cornell_pairs.csv")
df.head()
```

	input	response	
0	can we make this quick? roxanne korrine and an...	well, i thought we'd start with pronunciation,...	
1	well, i thought we'd start with pronunciation,...	not the hacking and gagging and spitting part...	
2	not the hacking and gagging and spitting part....	okay... then how 'bout we try out some french ...	
3	you're asking me out. that's so cute. what's y...	forget it.	
4	no, no, it's my fault -- we didn't have a prop...	cameron.	

Next steps:

[Generate code with df](#)
[New interactive sheet](#)

```
%%bash
cat > dataset_for_transformers.py << 'PY'
from torch.utils.data import Dataset
from transformers import AutoTokenizer
import pandas as pd
import torch

class ChatDataset(Dataset):
    def __init__(self, csv_path, tokenizer_name="microsoft/DialoGPT-small", max_length=128):
        self.df = pd.read_csv(csv_path)
        self.tokenizer = AutoTokenizer.from_pretrained(tokenizer_name)
        self.max_length = max_length
        if self.tokenizer.pad_token is None:
            self.tokenizer.pad_token = self.tokenizer.eos_token

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        prompt = str(row["input"]).strip()
        response = str(row["response"]).strip()
        enc_prompt = self.tokenizer.encode(prompt, add_special_tokens=False)
        enc_resp = self.tokenizer.encode(response, add_special_tokens=False)
        input_ids = enc_prompt + [self.tokenizer.eos_token_id] + enc_resp + [self.tokenizer.eos_t
        # truncate to rightmost tokens (keep response tail)
        if len(input_ids) > self.max_length:
            input_ids = input_ids[-self.max_length:]
        attention_mask = [1] * len(input_ids)
        prompt_len = len(enc_prompt) + 1
        labels = [-100] * prompt_len + input_ids[prompt_len:]
        # pad
        pad_len = self.max_length - len(input_ids)
        if pad_len > 0:
            input_ids = input_ids + [self.tokenizer.pad_token_id]*pad_len
            attention_mask = attention_mask + [0]*pad_len
```

```

        labels = labels + [-100]*pad_len
    return {
        "input_ids": torch.tensor(input_ids, dtype=torch.long),
        "attention_mask": torch.tensor(attention_mask, dtype=torch.long),
        "labels": torch.tensor(labels, dtype=torch.long)
    }
PY

```

```

%%bash
cat > train_dialo.py << 'PY'
import os
from transformers import AutoModelForCausalLM, Trainer, TrainingArguments
from dataset_for_transformers import ChatDataset
import torch

model_name = "microsoft/DialoGPT-small"
train_csv = "cornell_pairs.csv"
output_dir = "/content/drive/MyDrive/chatbot_models/dialo_finetuned" # save to Drive
max_length = 128

dataset = ChatDataset(train_csv, tokenizer_name=model_name, max_length=max_length)

model = AutoModelForCausalLM.from_pretrained(model_name)
model.config.pad_token_id = model.config.eos_token_id

training_args = TrainingArguments(
    output_dir=output_dir,
    overwrite_output_dir=True,
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4, # effective batch = 16
    num_train_epochs=2,
    fp16=torch.cuda.is_available(),
    save_steps=2000,
    save_total_limit=2,
    logging_steps=200,
    remove_unused_columns=False,
    learning_rate=5e-5,
    weight_decay=0.01
)

def collate_fn(batch):
    import torch
    input_ids = torch.stack([b["input_ids"] for b in batch])
    attention_mask = torch.stack([b["attention_mask"] for b in batch])
    labels = torch.stack([b["labels"] for b in batch])
    return {"input_ids": input_ids, "attention_mask": attention_mask, "labels": labels}

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=dataset,
    data_collator=collate_fn
)

trainer.train()
trainer.save_model(output_dir)
print("Model saved to", output_dir)
PY

```



```

# Robust inference helper for DialogPT / DialogPT-finetuned models
# Paste and run this in Colab. Edit model_dir variable if needed.

import os
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM

# 1) Mount drive if you saved model on Drive (uncomment if not mounted)
# from google.colab import drive
# drive.mount('/content/drive')

# 2) Set the model directory here.
# If you saved to Drive earlier, use the Drive path, e.g.,
# model_dir = "/content/drive/MyDrive/chatbot_models/dialo_finetuned"
# Or if you didn't fine-tune yet, set to "microsoft/DialogPT-small" to test pretrained model
model_dir = "/content/drive/MyDrive/chatbot_models/dialo_finetuned" # CHANGE this if needed
# model_dir = "microsoft/DialogPT-small" # fallback: use official pretrained

# 3) Quick existence check
print("Checking model directory:", model_dir)
if os.path.exists(model_dir):
    print("Directory exists. Listing files:")
    for root, dirs, files in os.walk(model_dir):
        # show only top-level few files
        if root == model_dir:
            print("FILES:", files)
            break
else:
    print("Directory does not exist. Will attempt to load from Hugging Face hub if model_dir is a
    # if model_dir is not local, the Auto* loaders will try hub

# 4) Load tokenizer and model safely
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Device:", device)

try:
    tokenizer = AutoTokenizer.from_pretrained(model_dir, use_fast=True)
    model = AutoModelForCausalLM.from_pretrained(model_dir)
    print("Loaded tokenizer and model from:", model_dir)
except Exception as e:
    print("Loading from local dir failed with error:", repr(e))
    print("Trying to load pretrained microsoft/DialogPT-small as fallback...")
    model_dir = "microsoft/DialogPT-small"
    tokenizer = AutoTokenizer.from_pretrained(model_dir, use_fast=True)
    model = AutoModelForCausalLM.from_pretrained(model_dir)
    print("Loaded fallback model:", model_dir)

# 5) Ensure pad token and eos token exist (common source of generation errors)
if tokenizer.pad_token is None:
    if tokenizer.eos_token is not None:
        print("Setting pad_token to eos_token for tokenizer.")
        tokenizer.pad_token = tokenizer.eos_token
    else:
        # rare: set pad_token to tokenizer.token_to_id(".") fallback
        tokenizer.add_special_tokens({'pad_token': '[PAD]'})
        print("Added a pad token '[PAD]' to tokenizer.")

if model.config.pad_token_id is None:
    model.config.pad_token_id = tokenizer.pad_token_id
    print("Set model.config.pad_token_id to", model.config.pad_token_id)

```



```

if model.config.eos_token_id is None and tokenizer.eos_token_id is not None:
    model.config.eos_token_id = tokenizer.eos_token_id
    print("Set model.config.eos_token_id to", model.config.eos_token_id)

model.to(device)
model.eval()
print("Model moved to device and set to eval()")

# 6) Inference function (safe defaults, small output to avoid OOM)
def chat_reply(user_text, max_new_tokens=50, top_k=50, top_p=0.95, temperature=0.8):
    # encode user prompt
    input_text = user_text.strip()
    if input_text == "":
        return ""
    try:
        input_ids = tokenizer.encode(input_text + tokenizer.eos_token, return_tensors="pt").to(device)
    except Exception as e:
        # fallback to tokenizer.__call__
        input_ids = tokenizer(input_text + tokenizer.eos_token, return_tensors="pt").input_ids.to(device)

    # generate
    with torch.no_grad():
        out = model.generate(
            input_ids,
            do_sample=True,
            max_new_tokens=max_new_tokens,
            top_k=top_k,
            top_p=top_p,
            temperature=temperature,
            pad_token_id=tokenizer.pad_token_id,
            eos_token_id=tokenizer.eos_token_id,
            num_return_sequences=1
        )

    # decode only new tokens
    gen_tokens = out[0][input_ids.shape[1]:]
    reply = tokenizer.decode(gen_tokens, skip_special_tokens=True).strip()
    return reply

# 7) Test on sample prompts
tests = [
    "hello, how are you?",
    "what is your name?",
    "tell me a joke",
    "how do i make spaghetti?"
]

print("\nSample replies:")
for t in tests:
    try:
        r = chat_reply(t)
    except Exception as e:
        r = f"ERROR during generation: {repr(e)}"
    print(f"> {t}\n-> {r}\n")

# 8) If you want interactive testing in the notebook:
print("You can now call chat_reply('your text') to test interactively.")

```



```

Checking model directory: /content/drive/MyDrive/chatbot_models/dialo_finetuned
Directory does not exist. Will attempt to load from Hugging Face hub if model_dir is a hub id.
Device: cpu
Loading from local dir failed with error: HFValidationError("Repo id must be in the form 'repo_name'")
Trying to load pretrained microsoft/DialogPT-small as fallback...
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

tokenizer_config.json: 100% 614/614 [00:00<00:00, 24.7kB/s]

vocab.json: 1.04M/? [00:00<00:00, 10.9MB/s]

merges.txt: 456k/? [00:00<00:00, 10.1MB/s]

config.json: 100% 641/641 [00:00<00:00, 15.8kB/s]

model.safetensors: 100% 351M/351M [00:03<00:00, 234MB/s]

generation_config.json: 100% 124/124 [00:00<00:00, 14.0kB/s]

The attention mask is not set and cannot be inferred from input because pad token is same as eos token.
Loaded fallback model: microsoft/DialogPT-small
Setting pad_token to eos_token for tokenizer.
Set model.config.pad_token_id to 50256
Model moved to device and set to eval()

Sample replies:
> hello, how are you?
-> I'm in the same boat, just got here.

> what is your name?
-> IGN is xDalley

> tell me a joke
-> I am sure the kid in the back is enjoying being with the mom.

> how do i make spaghetti?
-> how do i make memes?

You can now call chat_reply('your text') to test interactively.

```

Start coding or [generate](#) with AI.

